

EXPERIMENT 19

TITLE: WRITE A PROLOG PROGRAM FOR STUDENT-TEACHER-SUB-CODE

Aim

To develop a Prolog program to represent the relationship between students, teachers, and subject codes and retrieve the required information using logical queries.

Objective

- To understand the use of facts and rules in Prolog.
- To represent relationships between students, teachers, and subjects.
- To create a simple knowledge base using Prolog.
- To retrieve information using queries.
- To understand logical inference and relationship-based searching.

Theory

Prolog can be used to represent relationships between different entities using facts and rules. In this experiment, information about students, teachers, and subject codes is stored in a knowledge base.

The predicate `student_teacher_subject(Student, Teacher, SubjectCode)` represents the relationship between a student, a teacher, and the corresponding subject code.

For example:

```
student_teacher_subject(varshini, ravi, cs101).
```

This means that Varshini studies the subject with code CS101, which is handled by Ravi.

Prolog queries can be used to find the teacher of a particular subject, the subject code handled by a teacher, or all subjects associated with a student.

Algorithm

Step 1: Start the program.

Step 2: Define facts representing the student-teacher-subject relationships.

Step 3: Store the student name, teacher name, and subject code.

Step 4: Define rules to retrieve the required information.

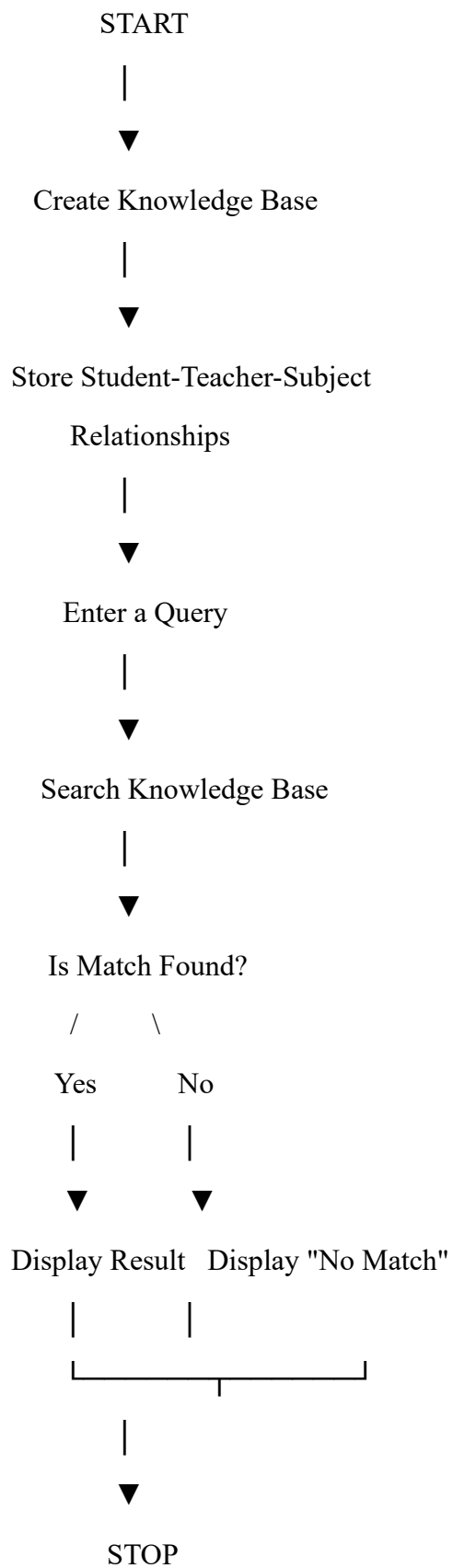
Step 5: Enter a query to search the knowledge base.

Step 6: Prolog matches the query with the available facts.

Step 7: Display the matching information.

Step 8: Stop the program.

Flowchart



Prolog Program

% Student-Teacher-Subject-Code Database

```
student_teacher_subject(varshini, ravi, cs101).  
student_teacher_subject(varshini, priya, cs102).  
student_teacher_subject(rahul, ravi, cs101).  
student_teacher_subject(rahul, kumar, cs103).  
student_teacher_subject(priya, priya, cs102).  
student_teacher_subject(arun, kumar, cs103).  
student_teacher_subject(kavya, meena, cs104).
```

% Rule to find teacher and subject code of a student

```
find_student_details(Student, Teacher, SubjectCode) :-  
    student_teacher_subject(Student, Teacher, SubjectCode).
```

% Rule to find students taught by a teacher

```
find_teacher_students(Teacher, Student, SubjectCode) :-  
    student_teacher_subject(Student, Teacher, SubjectCode).
```

Queries

Query 1: Find the teacher and subject code of Varshini

?- find_student_details(varshini, Teacher, SubjectCode).

Sample Output

Teacher = ravi,

SubjectCode = cs101 ;

Teacher = priya,

SubjectCode = cs102.

Query 2: Find the students taught by Ravi

?- find_teacher_students(ravi, Student, SubjectCode).

Sample Output

Student = varshini,

SubjectCode = cs101 ;

Student = rahul,

SubjectCode = cs101.

Query 3: Find the subject code handled by Priya

?- student_teacher_subject(Student, priya, SubjectCode).

Sample Output

Student = varshini,

SubjectCode = cs102 ;

Student = priya,

SubjectCode = cs102.

Query 4: Display all student-teacher-subject relationships

?- student_teacher_subject(Student, Teacher, SubjectCode).

Sample Output

Student = varshini,

Teacher = ravi,

SubjectCode = cs101 ;

Student = varshini,

Teacher = priya,

SubjectCode = cs102 ;

Student = rahul,

Teacher = ravi,

SubjectCode = cs101 ;

Student = rahul,
Teacher = kumar,
SubjectCode = cs103 ;

Student = priya,
Teacher = priya,
SubjectCode = cs102 ;

Student = arun,
Teacher = kumar,
SubjectCode = cs103 ;

Student = kavya,
Teacher = meena,
SubjectCode = cs104.

Explanation

The predicate student_teacher_subject/3 contains three arguments:

Student, Teacher, SubjectCode

For example:

student_teacher_subject(varshini, ravi, cs101).

represents the relationship between Varshini, Ravi, and subject code CS101.

The rule:

find_student_details(Student, Teacher, SubjectCode) :-

student_teacher_subject(Student, Teacher, SubjectCode).

is used to retrieve the teacher and subject code associated with a particular student.

Similarly, the rule:

find_teacher_students(Teacher, Student, SubjectCode) :-

student_teacher_subject(Student, Teacher, SubjectCode).

can be used to find students associated with a particular teacher.

Advantages

- Simple representation of relationships.
- Easy retrieval using logical queries.
- Supports multiple relationships.
- Demonstrates knowledge representation in Prolog.
- Useful for database-like applications.

Applications

- College management systems.
- Student information systems.
- Timetable management.
- Teacher-subject allocation systems.
- Knowledge-based systems.
- Artificial Intelligence applications.

Result

The Prolog program was successfully implemented to represent the relationship between students, teachers, and subject codes. The required information was successfully retrieved using Prolog queries.

Conclusion

Thus, the STUDENT-TEACHER-SUB-CODE relationship was successfully implemented using Prolog. This experiment demonstrates how Prolog facts, rules, and queries can be used to represent and retrieve relationships in a knowledge base.

--- PLANETS DATABASE ---

1. Display all planets

2. Search for a planet

3. Exit

Enter your choice: 2

Enter planet name: Earth

Planet: Earth

Type: Terrestrial

Distance from Sun: 149.6 million km

Number of Moons: 1

```

5
6
7 # Display all planets
8 def display_planets():
9     print("\nPLANETS DATABASE")
10    print("-" * 55)
11
12    for planet, details in planets.items():
13        print("Planet:", planet)
14        print("Type:", details["type"])
15        print("Distance from Sun:", details["distance"], "million km")
16        print("Number of Moons:", details["moons"])
17        print("-" * 55)
18
19
20 # Search for a planet
21 def search_planet(name):
22     if name in planets:
23         details = planets[name]
24
25         print("\nPlanet:", name)
26         print("Type:", details["type"])
27         print("Distance from Sun:", details["distance"], "million km")
28         print("Number of Moons:", details["moons"])
29     else:
30         print("Planet not found.")
31
32
33 # Main program
34 while True:
35     print("\n--- PLANETS DATABASE ---")
36     print("1. Display all planets")
37     print("2. Search for a planet")
38     print("3. Exit")
39
40     choice = input("Enter your choice: ")
41
42     if choice == "1":
43         display_planets()
44
45     elif choice == "2":
46         name = input("Enter planet name: ").strip().capitalize()
47         search_planet(name)
48
49     elif choice == "3":
50         print("Program terminated.")
51         break
52
53     else:
54         print("Invalid choice.")

```


C:\> Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs > EXP1.py > ...

```
1  # PLANETS DATABASE
2
3  planets = {
4      "Mercury": {
5          "type": "Terrestrial",
6          "distance": 57.9,
7          "moons": 0
8      },
9      "Venus": {
10         "type": "Terrestrial",
11         "distance": 108.2,
12         "moons": 0
13     },
14     "Earth": {
15         "type": "Terrestrial",
16         "distance": 149.6,
17         "moons": 1
18     },
19     "Mars": {
20         "type": "Terrestrial",
21         "distance": 227.9,
22         "moons": 2
23     },
24     "Jupiter": {
25         "type": "Gas Giant",
26         "distance": 778.5,
27         "moons": 95
28     },
29     "Saturn": {
30         "type": "Gas Giant",
31         "distance": 1434,
32         "moons": 146
33     },
34     "Uranus": {
35         "type": "Ice Giant",
36         "distance": 2871,
37         "moons": 28
38     },
39     "Neptune": {
40         "type": "Ice Giant",
41         "distance": 4495,
42         "moons": 16
43     }
44 }
45
46
```